

Course Number	COE 328
Course Title	Digital Systems
Semester/Year	Semester 3, Year 2
Section Number	1

Lab - Circuit & Storage Elements

Assignment Title	Lab 5
Submission Date	November 19th, 2022
Due Date	November 20th, 2022

Student Name	Student ID (xxxx1234)	Signature*
Ahmad Najafi	24585	Najafi
Syed Maisam Abbas	03255	Maisam

(Note: Remove the first 4 digits from your student ID)

**By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <http://www.ryerson.ca/senate/policies/pol60.pdf>.*

Pre-Lab

For lab 5 the chosen student id $D = \{d1, d2, d3, d4, d5, d6, d7, d8, d9\} = \{5, 0, 1, 1, 0, 3, 2, 5, 5\}$ where the modulus is $3255/13 = 5$. Therefore, we were assigned FSM 5 and chose the moore machine.

Present State	Next State		Output	
	w = 0	w = 1	w = 0	w = 1
S ₀	S ₀	S ₁	5	0
S ₁	S ₁	S ₈	0	5
S ₈	S ₈	S ₂	5	1
S ₂	S ₂	S ₇	1	5
S ₇	S ₇	S ₃	5	1
S ₃	S ₃	S ₆	1	2
S ₆	S ₆	S ₄	2	0
S ₄	S ₄	S ₅	0	3
S ₅	S ₅	S ₀	3	5

Table 1: State Table

Present State				Next State								Output							
y ₃	y ₂	y ₁	y ₀	w = 0				w = 1				w = 0				w = 1			
0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	1	0	0	0	0
0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	0	0	1	0	1
0	0	1	0	1	0	0	0	0	0	1	0	0	1	0	1	0	0	0	1
0	0	1	1	0	0	1	0	0	1	1	1	0	0	0	1	0	1	0	1
0	1	0	0	0	1	1	1	0	0	1	1	0	1	0	1	0	0	0	1
0	1	0	1	0	0	1	1	0	1	1	0	0	0	0	1	0	0	1	0
0	1	1	0	0	1	1	0	0	1	0	0	0	0	1	0	0	0	0	0
0	1	1	1	0	1	0	0	0	1	0	1	0	0	0	0	0	0	1	1
1	0	0	0	0	1	0	1	0	0	0	0	0	0	1	1	0	1	0	1
1	0	0	1	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
1	0	1	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
1	0	1	1	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
1	1	0	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
1	1	0	1	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
1	1	1	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
1	1	1	1	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x

Table 2: Assigned State Table

Student_id_digit_1	d_1	5
Student_id_digit_2	d_2	0
Student_id_digit_3	d_3	1
Student_id_digit_4	d_4	1
Student_id_digit_5	d_5	0
Student_id_digit_6	d_6	3
Student_id_digit_7	d_7	2
Student_id_digit_8	d_8	5
Student_id_digit_9	d_9	5

Lab 5

$y_3:$

$y_2 y_1 y_0$									
$w y_3$		000	001	010	011	100	101	110	111
00		0	0	0	0	0	0	0	0
01		0	0	0	0	0	0	0	0
11		0	0	0	0	0	0	0	0
10		0	0	0	0	0	0	0	0

$y_3 = 0$

$y_2:$

$y_2 y_1 y_0$									
$w y_3$		000	001	010	011	100	101	110	111
00		①		①		①			
01									
11		①	①						
10			①		①				

$$y_2 = \overline{w} \overline{y_3} \overline{y_2} \overline{y_1} \overline{y_0} + \overline{w} \overline{y_3} \overline{y_2} \overline{y_1} \overline{y_0} + \overline{w} \overline{y_3} \overline{y_2} \overline{y_1} \overline{y_0} + \overline{w} \overline{y_3} \overline{y_2} \overline{y_1} \overline{y_0} + \dots$$

$$\dots + \overline{w} \overline{y_3} \overline{y_2} \overline{y_1} \overline{y_0} + \overline{w} \overline{y_3} \overline{y_2} \overline{y_1} \overline{y_0}$$

$Y_1:$

$y_2 y_1 y_0$		000	001	010	011	100	101	110	111
$w y_3$	00					1		①	
	01	①				1			
	11								1
	10				1		①		①

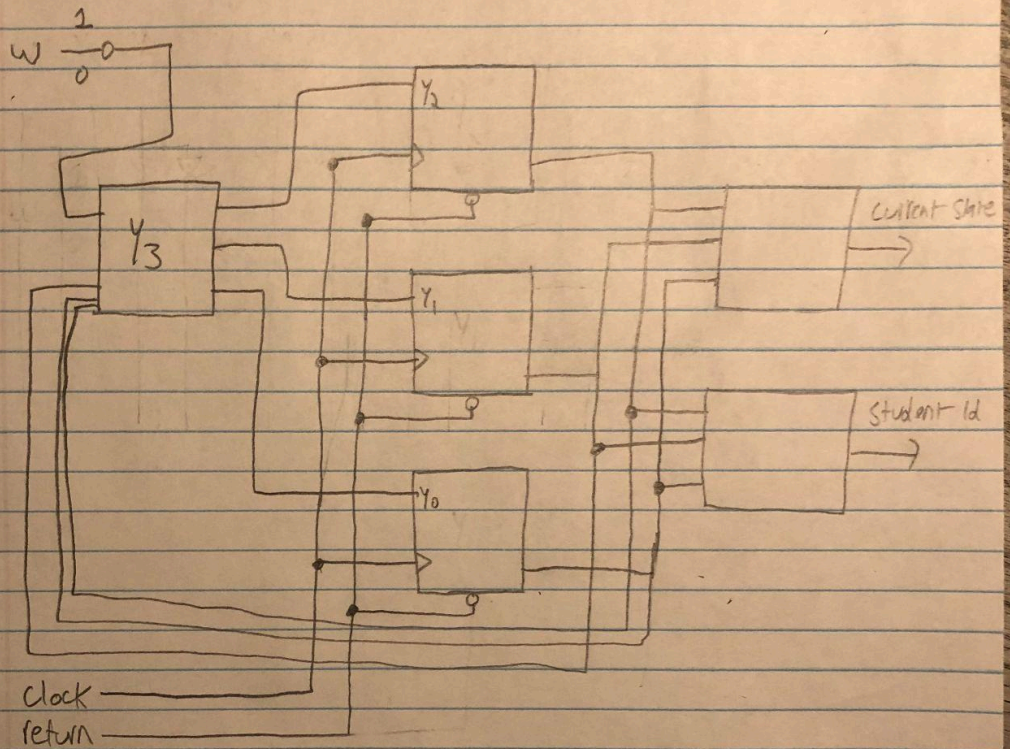
$$Y_1 = \bar{w} \bar{y}_3 y_2 y_1 \bar{y}_0 + \bar{w} y_3 \bar{y}_2 \bar{y}_1 \bar{y}_0 + w \bar{y}_3 y_2 \bar{y}_1 y_0 + w \bar{y}_3 y_2 y_1 y_0$$

$Y_0:$

$y_2 y_1 y_0$		000	001	010	011	100	101	110	111
$w y_3$	00	1		1	1	1	1		
	01	①							
	11	1	1	1	1	1			
	10		1	1	1	1			①

$$Y_0 = \bar{w} \bar{y}_3 + w \bar{y}_3 + w \bar{y}_3 y_2 y_1 y_0 + y_3 \bar{y}_2 \bar{y}_1 \bar{y}_0 + \bar{w} \bar{y}_2 \bar{y}_1 \bar{y}_0$$

Logic diagram



Post-lab:

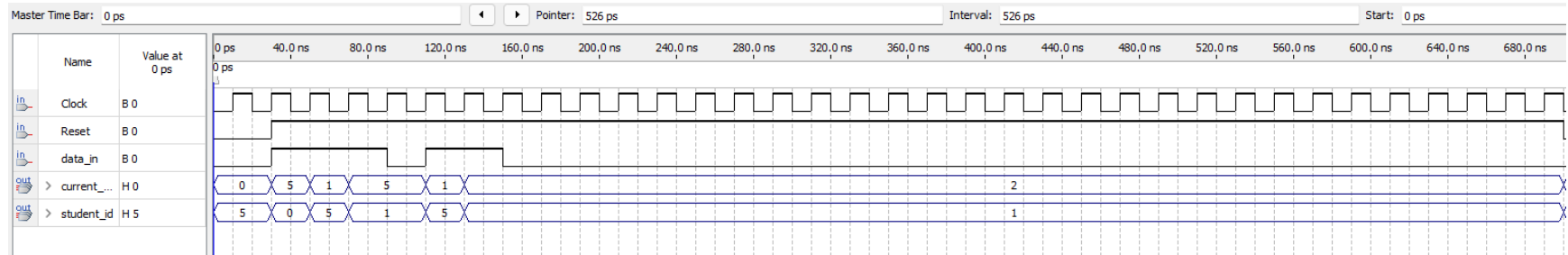
```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity Lab5 is
5  port (
6      Clock, Reset : in std_logic;
7      data_in : in std_logic;
8      student_id : out std_logic_vector(3 downto 0);
9      current_state : out std_logic_vector(3 downto 0)
10 );
11 end Lab5;
12
13 architecture Behavior of Lab5 is
14     type state_type is(s0, s1, s2, s3, s4, s5, s6, s7, s8);
15     signal y: state_type;
16     --signal control_signal: std_logic_vector(9 downto 0);
17 begin
18     process(Clock, Reset)
19     begin
20         if (Reset = '0') then
21             y <= s0;
22         elsif (Clock'EVENT and Clock = '1') then
23             case y is
24                 when s0 =>
25                     case data_in is
26                         when '0' => y <= s0;
27                         when '1' => y <= s1;
28                         when others => y <= s0;
29                     end case;
30                 when s1 =>
31                     case data_in is
32                         when '0' => y <= s1;
33                         when '1' => y <= s8;
34                         when others => y <= s1;
35                     end case;
36                 when s2 =>
37                     case data_in is
38                         when '0' => y <= s2;
39                         when '1' => y <= s7;
40                         when others => y <= s2;
41                     end case;
42                 when s3 =>
43                     case data_in is
44                         when '0' => y <= s3;
45                         when '1' => y <= s6;
46                         when others => y <= s3;
47                     end case;
48                 when s4 =>
49                     case data_in is
50                         when '0' => y <= s4;
51                         when '1' => y <= s5;
52                         when others => y <= s4;
53                     end case;
54                 when s5 =>
55                     case data_in is
56                         when '0' => y <= s5;
57                         when '1' => y <= s0;
58                         when others => y <= s5;
59                     end case;
```

```

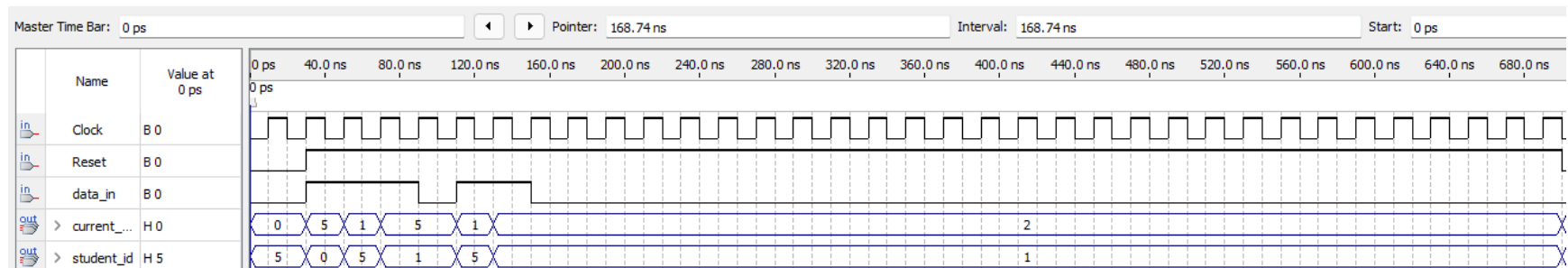
60         when s6 =>
61             case data_in is
62                 when '0' => y <= s6;
63                 when '1' => y <= s4;
64                 when others => y <= s6;
65             end case;
66         when s7 =>
67             case data_in is
68                 when '0' => y <= s7;
69                 when '1' => y <= s3;
70                 when others => y <= s7;
71             end case;
72         when s8 =>
73             case data_in is
74                 when '0' => y <= s8;
75                 when '1' => y <= s2;
76                 when others => y <= s8;
77             end case;
78         end case;
79     end if;
80 end process;
81
82 process (y, data_in)
83 begin
84     case y is
85         when s0 =>
86             student_id <= X"5";
87             current_state <= X"0";
88         when s1 =>
89             student_id <= X"0";
90             current_state <= X"5";
91         when s2 =>
92             student_id <= X"1";
93             current_state <= X"5";
94         when s3 =>
95             student_id <= X"1";
96             current_state <= X"2";
97         when s4 =>
98             student_id <= X"0";
99             current_state <= X"3";
100        when s5 =>
101            student_id <= X"3";
102            current_state <= X"5";
103        when s6 =>
104            student_id <= X"2";
105            current_state <= X"0";
106        when s7 =>
107            student_id <= X"5";
108            current_state <= X"1";
109        when s8 =>
110            student_id <= X"5";
111            current_state <= X"1";
112        when others =>
113            student_id <= X"D";
114            current_state <= X"D";
115        end case;
116    end process;
117 end Behavior;

```

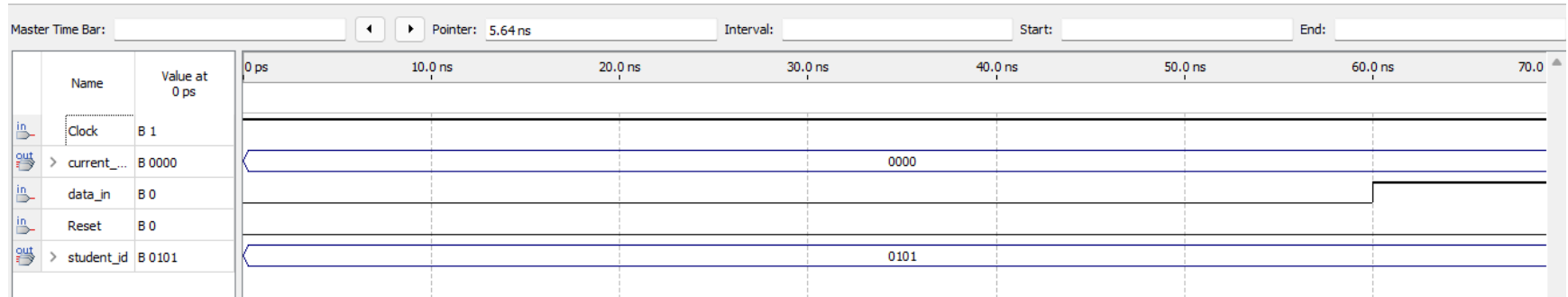
Test 1:



Test 2:



Test 3:



Based on the given inputs of each simulation test, the output represents a new formation provided from the given FSM diagram through the use of the VHDL code. The clock goes through a continuous cycle, data_in accounts for the given input, and reset is set to zero for the 3/2 of the first period and switched on for the rest. The outputs display the current state and the student ID.

```
library ieee;
use ieee.std_logic_1164.all;
entity FSM is
    port(
        clk          :in std_logic;
        data_in       :in std_logic;
        reset         :in std_logic;
        student_id     :out  std_logic_vector(3 downto 0);
        current_state :out std_logic_vector(3 downto 0));
end entity;

architecture fsm of FSM is
    type state_type is (s0, s1, s2, s3, s4, s5, s6, s7, s8);

    signal yfsm: state_type;
begin
    process(clk, reset)
    begin
        if reset = '1' then -- resets the states to initial states
            yfsm <= s0;
        elsif (clk 'EVENT AND clk = '1') then
            case yfsm is
                when s0 =>
                    if (data_in = '1') then
                        yfsm <= s1;
                    else
                        yfsm <= s0;
                    end if;

                when s1 =>
                    if (data_in = '1') then
                        yfsm <= s8;
                    else
```

```
        yfsm <= s1;  
    end if;
```

```
when s2 =>  
    if(data_in = '1')then  
        yfsm <= s7;  
    else  
        yfsm <= s2;  
    end if;
```

```
when s3 =>  
    if(data_in = '1')then  
        yfsm <= s6;  
    else  
        yfsm <= s3;  
    end if;
```

```
when s4 =>  
    if(data_in = '1')then  
        yfsm <= s5;  
    else  
        yfsm <= s4;  
    end if;
```

```
when s5 =>  
    if(data_in = '1')then  
        yfsm <= s0;  
    else  
        yfsm <= s5;  
    end if;
```

```
when s6 =>  
    if(data_in = '1')then  
        yfsm <= s4;  
    else  
        yfsm <= s6;  
    end if;
```

```
when s7 =>  
    if(data_in = '1')then
```

```

                                yfsm <= s3;
                                else
                                yfsm <= s7;
                                end if;

                                when s8 =>
                                if(data_in = '1')then
                                yfsm <= s2;
                                else
                                yfsm <= s8;
                                end if;
                                end case;
                                end if;
end process;

process(yfsm, data_in)
begin
case yfsm is
when s0 =>
    current_state <= "0000";
    student_id <= "0101";

when s1 =>
    current_state <= "0001";
    student_id <= "0000";

when s2 =>
    current_state <= "0010";
    student_id <= "0001";

when s3 =>
    current_state <= "0011";
    student_id <= "0001";

when s4 =>
    current_state <= "0100";
    student_id <= "0000";

when s5 =>
    current_state <= "0101";

```

```
        student_id <= "0011";

    when s6 =>
        current_state <= "0110";
        student_id <= "0010";

    when s7 =>
        current_state <= "0111";
        student_id <= "0101";

    when s8 =>
        current_state <= "1000";
        student_id <= "0101";

    end case;
end process;
end Architecture;
```